

BIRZEIT UNIVERSITY

Department of Electrical & Computer Engineering
ENCS4110 – Computer Design Laboratory

Experiment 03

Branching and Loops

Date: October 2026

3 Branching and Loops

Learning Objectives

After completing this experiment, you will be able to:

- Set the condition flags with compare and test instructions (`CMP`, `CMN`, `TST`, `TEQ`).
- Implement conditional and unconditional branching using ARM branch instructions.
- Choose the correct signed or unsigned condition code for a comparison.
- Use conditional execution (and the `IT` block behind it) to avoid short branches.
- Design and implement loop constructs (`for`, `while`) that process arrays and null-terminated strings.

Experiment Overview

This experiment introduces program flow control on the ARM Cortex-M4 processor. You will compare values and test bits to set the condition flags, use conditional branches and conditional execution to make decisions, and combine them into `for` and `while` loops that walk through arrays and strings. By the end of this lab, you will be able to translate high-level `if`, `for`, and `while` constructs into assembly. Procedures and the stack build on this in Experiment 4.

3.1 Theoretical Background

3.1.1 Flow Control

Flow control instructions alter the sequential execution of instructions by changing the program counter (PC). These instructions enable the implementation of conditional statements, loops, and procedure calls that are fundamental to structured programming.

3.1.1.1 Condition Evaluation

Before implementing control flow, we must understand how to evaluate conditions and set processor flags. ARM provides dedicated instructions for comparing values and testing bit patterns that update the condition flags without storing results.

Table 3.1: ARM Cortex-M4 Compare and Test Instructions

Instr.	Syntax	Description / Usage
CMP	`CMP Rn, Operand2`	Compare `Rn` with `Operand2` (`Rn - Operand2`); updates flags (Z, N, C, V).
CMN	`CMN Rn, Operand2`	Compare negative: computes `Rn + Operand2` and sets flags as if by `ADD`, i.e. tests whether `Rn == -Operand2`.
TST	`TST Rn, Operand2`	Logical AND test (`Rn AND Operand2`); sets Z if all tested bits are 0.
TEQ	`TEQ Rn, Operand2`	Logical XOR test (`Rn EOR Operand2`); sets Z if operands are equal.

Usage Notes:

- These instructions only affect the condition flags (N, Z, C, V) — they do not store a result.
- `CMP`/`CMN` are arithmetic comparisons; `TST`/`TEQ` are logical bitwise comparisons.
- Many data-processing instructions can update flags by appending `S` (e.g., `ADDS`, `SUBS`).
- Common use: immediately followed by conditional branches such as `BEQ`, `BNE`, `BGT`, etc.

Examples:

```
CMP     R0, #10          ; Compare R0 with 10
BLT     LessThan10       ; Branch if R0 < 10
BGE     GreaterOrEqual   ; Otherwise, R0 >= 10
```

Listing 3.1: Arithmetic comparison using CMP

```
MOV     R1, #0x12        ; R1 = 0001 0010b
TST     R1, #0x10        ; Test if bit 4 is set
BEQ     BitClear         ; Branch if bit 4 = 0 (Z=1)
BNE     BitSet           ; Branch if bit 4 = 1 (Z=0)
```

Listing 3.2: Bit test using TST

```
MOV     R2, #0x55
MOV     R3, #0x55
TEQ     R2, R3           ; XOR -> result 0, sets Z=1
BEQ     ValuesEqual      ; Branch if equal
```

Listing 3.3: Equality check using TEQ

3.1.1.2 Conditional Branching

Branch instructions are the primary mechanism for implementing flow control in ARM assembly. They modify the program counter to jump to different parts of the code based on conditions or unconditionally.

Table 3.2: ARM Cortex-M4 Branch Instructions

Instr.	Syntax	Description / Usage
B	`B label`	Unconditional branch to `label` (always jumps)
B<cond>	`B<cond> label`	Conditional branch based on flags
BL	`BL label`	Branch with link: calls a subroutine, storing return address in `LR`. Procedures are covered in Experiment 4.
BX	`BX Rm`	Branch to address in register, often `BX LR` to return from a subroutine.
CBZ	`CBZ Rn, label`	Branch if `Rn == 0`. Example: `CBZ R0, Done`.
CBNZ	`CBNZ Rn, label`	Branch if `Rn != 0`. Example: loop until counter reaches zero.

`CBZ/CBNZ` instructions have specific constraints:

- **Register**: operand must be a low register `R0-R7`.
- **Range**: branch is *forward-only*; the destination must be within 0–126 bytes after the instruction.
- **Flags**: does not update condition flags (`N, Z, C, V`).

For backward or longer jumps, use `CMP/TST` with conditional branches (`BEQ`, `BNE`, `BGT`, ...).

3.1.1.3 Conditional Execution

ARM assembly supports conditional execution, where most instructions can be conditionally executed based on the current state of the condition flags. This feature allows for efficient implementation of conditional statements without explicit branching.

Conditional Instruction Format: Most ARM instructions can be made conditional by appending a condition code suffix:

```
OPCODE{<cond>} Rd, Rn, Operand2
```

Examples:

- `ADDEQ R0, R1, R2` — Add only if equal (Z=1)
- `MOVNE R3, #10` — Move only if not equal (Z=0)
- `SUBGT R4, R4, #1` — Subtract only if greater than (signed)

Advantages of Conditional Execution:

- **Performance**: Eliminates branch instructions for simple conditional operations
- **Code density**: Reduces the number of instructions needed
- **Atomic operations**: Multiple related conditional operations can be grouped

The IT block: Cortex-M4 executes only Thumb-2 code, where a condition suffix on a non-branch instruction (`MOVGT`, `ADDEQ`, ...) is valid only inside an `IT` (If-Then) block. `IT` makes the next one to four instructions conditional: each one is marked `T` (execute if the condition holds) or `E` (execute if it does not), and every instruction in the block must carry the matching condition suffix.

```
CMP     R3, R2
ITE     GT              ; next 2 instructions: Then, Else
MOVGT   R2, R3          ; executes if R3 > R2
MOVLE   R4, #0          ; executes otherwise
```

Listing 3.4: Explicit IT block

If you write a conditional instruction without an `IT`, the Keil assembler normally inserts one for you. Look for it in the Disassembly window when you single-step Example 1 below. Conditional *branches* (`BEQ`, `BGT`, ...) are encoded differently and never need an `IT`.

3.1.1.4 How Branch Instructions Work

Branch instructions change the flow of execution by modifying the Program Counter (`PC`). When a branch is executed, the instruction encodes an *offset* which is added to the current value of the `PC`.

Offset calculation: The branch instruction contains a signed immediate value (positive or negative). The processor adds this offset (aligned to halfword boundaries) to the current `PC`.

- A *positive offset* causes a **forward branch** (jump to a higher memory address, later in the program).
- A *negative offset* causes a **backward branch** (jump to a lower memory address, earlier in the program).

Example: Suppose a branch instruction is located at address `0x100`, and the assembler encodes an immediate offset of `-0x08`. The effective target address will be:

$$0x100 + 4 + (-0x08) = 0xFC$$

This means the processor jumps **backward** to an earlier instruction. Such negative offsets are typically used to implement loops (e.g., repeat until zero).

3.1.1.5 Condition Codes

Conditional branches use condition codes that test the processor status flags (N, Z, C, V) set by previous instructions. These enable implementing high-level constructs like if-statements and loops.

Table 3.3: Common ARM Condition Codes

Cond.	Meaning	Description
EQ	Equal	Execute if $Z = 1$.
NE	Not equal	Execute if $Z = 0$.
CS/HS	Carry set / Unsigned higher or same	Execute if $C = 1$.
CC/LO	Carry clear / Unsigned lower	Execute if $C = 0$.
MI	Minus (negative)	Execute if $N = 1$.
PL	Plus (non-negative)	Execute if $N = 0$.
VS	Overflow set	Execute if $V = 1$.
VC	Overflow clear	Execute if $V = 0$.
HI	Unsigned higher	Execute if $C = 1$ *and* $Z = 0$.
LS	Unsigned lower or same	Execute if $C = 0$ *or* $Z = 1$.
GE	Greater or equal (signed)	Execute if $N = V$.
LT	Less than (signed)	Execute if $N \neq V$.
GT	Greater than (signed)	Execute if $Z = 0$ *and* $N = V$.
LE	Less or equal (signed)	Execute if $Z = 1$ *or* $N \neq V$.
AL	Always	Always execute (default if no condition).
NV	Never	Reserved / do not use.

3.1.2 Loop Patterns

Loops are fundamental control structures that repeat a block of code based on conditions. ARM assembly implements loops using combinations of compare instructions, conditional branches, and counters.

3.1.2.1 For Loop Structure

A typical for loop has the structure: initialization, condition check, body execution, and increment/decrement. This type of loop executes a known number of times.

```
        AREA M_DATA, DATA, READONLY
array   DCD 10, 20, 30, 40, 50    ; array of 5 integers
length  EQU 5                     ; number of elements (; just a constant, no memory)
```

Listing 3.5: Declaring Array and Length

```
        ; Initialization
        MOV     R0, #0                  ; i = 0
        MOV     R1, #0                  ; sum = 0
        LDR     R3, =array              ; load base address of array into R3

for_start
        ; Condition check
        CMP     R0, #length             ; compare i with length
        BGE     for_end                 ; if i >= length, exit loop

        ; Loop body: scaled register-offset addressing  effective address
        ; is Rn + (Rm shifted), here R3 + (R0 * 4); this is a distinct
        ; addressing mode from the plain register-offset form covered in
```

```
        ; Experiment 2's addressing-modes table
        LDR     R2, [R3, R0, LSL #2]    ; load array[i]; EA: R3 + (R0 * 4)
        ADD     R1, R1, R2              ; sum += array[i]

        ; Increment
        ADD     R0, R0, #1              ; i++
        B       for_start               ; repeat

for_end
```

Listing 3.6: For loop implementation pattern

3.1.2.2 While Loop Structure

While loops check the condition before executing the loop body, potentially executing zero times if the initial condition is false. This type of loop is useful when the number of iterations is not known in advance and depends on dynamic conditions.

```
        AREA M_DATA, DATA, READONLY
mystring DCB "Hello World!", 0    ; null-terminated string
```

Listing 3.7: Declaring Null-Terminated String

Note: `0` and `'0'` are two different values, as the former is actually zero, while the latter is the ASCII code for the character '0' (which is 48 in decimal).

```
    ; Intialization
    LDR     R0, =mystring   ; pointer to string
    MOV     R1, #0          ; character count = 0

while_start
                            ; Condition check
    LDRB    R2, [R0], #1    ; load current character and post-increment pointer
    CMP     R2, #0          ; check for null terminator
    BEQ     while_end       ; if zero, exit loop

                            ; Loop body - do something with R2


    B       while_start     ; repeat
while_end
```

Listing 3.8: While loop with string processing example

3.2 Procedure

3.2.1 Examples

3.2.1.1 Example 1: Array Example — Find Maximum Element

This example demonstrates how to find the maximum element in an array using a standard for loop structure.

```
        AREA    RESET, CODE, READONLY
        EXPORT  __Vectors
__Vectors
        DCD     0x20001000          ; Initial SP
        DCD     Reset_Handler       ; Reset vector
        ALIGN

        AREA    MYCODE, CODE, READONLY
        ENTRY
        EXPORT  Reset_Handler

; Find maximum element in ARR[0..LEN-1]
; Result (max) is written to MAXRES.
Reset_Handler
        LDR     R0, =ARR            ; R0 = &ARR[0]
        MOV     R1, #0              ; R1 = i (index)
        LDR     R2, [R0]            ; R2 = max = ARR[0]
for_start
        CMP     R1, #LEN            ; i >= LEN ?
        BGE     for_end             ; yes -> done

        LDR     R3, [R0, R1, LSL #2]; R3 = ARR[i]
        CMP     R3, R2              ; if ARR[i] > max
        MOVGT   R2, R3              ;    max = ARR[i]
        ADD     R1, R1, #1          ; i++
        B       for_start
for_end
        LDR     R4, =MAXRES         ; store result for easy checking
        STR     R2, [R4]
STOP    B       STOP

        AREA    CONSTANTS, DATA, READONLY
ARR     DCD     10, 20, 30, -5, 11, 0
LEN     EQU     6

        AREA    MYDATA, DATA, READWRITE
MAXRES  DCD     0                   ; expect 30

        END
```

Listing 3.9: Find maximum element in an array

Check: Verify that the maximum element is correctly identified and stored in `MAXRES`.

3.2.1.2 Example 2: String Example — Count Uppercase Letters

This example demonstrates how to process a null-terminated string and count the number of uppercase letters (A-Z) using a while-loop structure.

```
        AREA    RESET, CODE, READONLY
        EXPORT  __Vectors
__Vectors
        DCD     0x20001000          ; Initial SP
        DCD     Reset_Handler       ; Reset vector
        ALIGN

        AREA    MYCODE, CODE, READONLY
        ENTRY
        EXPORT  Reset_Handler

; Count uppercase ASCII letters in the null-terminated string MYSTR.
; Result (count) is written to UPPERCOUNT.
Reset_Handler
        LDR     R0, =MYSTR          ; R0 = ptr to string
        MOV     R1, #0              ; R1 = count

while_next
        LDRB    R2, [R0], #1        ; R2 = *p++; post-increment pointer
        CBZ     R2, while_end       ; if '\0' -> exit

        CMP     R2, #'A'            ; below 'A'?
        BLT     while_next
        CMP     R2, #'Z'            ; above 'Z'?
        BGT     while_next

        ADD     R1, R1, #1          ; count++

        B       while_next

while_end
        LDR     R3, =UPPERCOUNT
        STR     R1, [R3]

STOP    B       STOP

        AREA    CONSTANTS, DATA, READONLY
MYSTR   DCB     "Hello ARM World!", 0

        AREA    MYDATA, DATA, READWRITE
UPPERCOUNT DCD  0                   ; expect 5 ('H','A','R','M','W')

        END
```

Listing 3.10: Count uppercase letters in a string

Check: Verify that the program correctly counts the uppercase letters and stores the result in `UPPERCOUNT`.

3.2.2 Tasks

3.2.2.1 Task 1: Count Vowels and Non-Vowels in a String

Write a program that scans the following null-terminated string, stored with `DCB`:

```
MYSTR   DCB  "ARM assembly language is important to learn!", 0
```

- Use R0 as the pointer to the current character.
- Count vowels (a, e, i, o, u, *upper- or lowercase*) in R1.
- Count every other character (consonants, spaces, and punctuation, but not the terminating `0`) in R2.
- Run the program and record the final values of R1 and R2. As a self-check, R1 + R2 must equal the string length.

Hint: Converting a letter to lowercase first (`ORR R3, R3, #0x20`) halves the number of comparisons. Think about why this is safe for letters, and why it doesn't turn a space or `!` into a vowel.

3.2.2.2 Task 2: Signed Array Statistics

Given the array of signed words:

```
ARR     DCD  7, -3, 12, -8, 0, 5, -1
LEN     EQU  7
```

Using a single loop over the array, compute:

- the sum of all elements in R1,
- the minimum element in R2,
- the number of negative elements in R3.

Then change the condition you used to update the minimum from the signed form (`LT`/`GE`) to the unsigned form (`LO`/`HS`) and run the program again. Report the new value of R2 and explain why it changed.